Reto 5 – Automatización de Pruebas Funcionales con BDD

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. Hasta ahora, se ha construido un sistema distribuido con:

- Servicios de negocio con persistencia y comunicación REST (Retos 1 y 2)
- Comunicación asincrónica basada en eventos (Reto 3)
- Seguridad centralizada con JWT y RBAC (Reto 4)

A lo largo de los retos anteriores, cada equipo ha verificado el funcionamiento del sistema ejecutando peticiones manualmente con herramientas como `curl` o Postman, siguiendo los flujos de prueba sugeridos paso a paso. Sin embargo, estas verificaciones no son repetibles, no están documentadas de forma ejecutable, y cada vez que se modifica un servicio, hay que repetir todo el proceso desde cero.

En este reto se aplicará el **Desarrollo Guiado por Comportamiento (BDD – Behavior-Driven Development)** como metodología para automatizar la validación funcional del sistema. Mediante escenarios escritos en **lenguaje natural**, se describirá el comportamiento esperado del sistema y se automatizará su verificación.

Objetivo

Implementar una suite de **pruebas funcionales automatizadas** utilizando la metodología **BDD** y herramientas como **Cucumber**. Los escenarios deben describir los flujos de negocio del sistema en lenguaje natural (Gherkin) y ejecutarse automáticamente contra el sistema completo desplegado con Docker Compose.

Introducción

¿Qué es BDD?

Behavior-Driven Development (BDD) es una metodología que extiende el desarrollo guiado por pruebas (TDD) al enfocar las pruebas en el **comportamiento** del sistema desde la perspectiva del usuario, no en la implementación técnica.

Aspecto	Pruebas Tradicionales	BDD
Perspectiva	Técnica (funciones, clases, módulos)	Negocio (flujos, comportamiento, reglas)
Lenguaje	Código del lenguaje de programación	Lenguaje natural estructurado (Gherkin)
Audiencia	Desarrolladores	Desarrolladores, QA, stakeholders
Documentación	Se desactualiza fácilmente	**Documentación viva**: si la prueba pasa, el comportamiento documentado es real
Pregunta clave	¿Funciona esta función?	¿El sistema se comporta correctamente?

El ciclo BDD sigue estas fases:

1. **Descubrir** — Identificar escenarios a partir de las historias de usuario
2. **Formular** — Escribir los escenarios en **Gherkin** (lenguaje natural estructurado)
3. **Automatizar** — Implementar los **step definitions** (código que ejecuta cada paso)
4. **Ejecutar** — Correr las pruebas; si fallan, desarrollar la funcionalidad
5. **Iterar** — Repetir para cada nueva funcionalidad o cambio

Sintaxis Gherkin

Gherkin es el lenguaje utilizado para escribir escenarios BDD. Se estructura con palabras clave que describen el contexto, la acción y el resultado esperado.

Palabras clave

Gherkin soporta múltiples idiomas de forma nativa. Para este reto, se utilizarán las palabras clave en español (activando `# language: es` en la primera línea del archivo `.feature`):

Inglés	Español
`Feature`	`Característica`
`Scenario`	`Escenario`
`Scenario Outline`	`Esquema del escenario`
`Given`	`Dado` / `Dada` / `Dados` / `Dadas`
`When`	`Cuando`
`Then`	`Entonces`
`And`	`Y`
`But`	`Pero`
`Examples`	`Ejemplos`
`Background`	`Antecedentes`

Estructura de un archivo `.feature`

- **Característica (Feature):** Funcionalidad o historia de usuario que se está probando.
- **Antecedentes (Background):** Precondiciones comunes a todos los escenarios del archivo.
- **Escenario (Scenario):** Un criterio de aceptación específico, con pasos `Dado/Cuando/Entonces`.
- **Esquema del escenario (Scenario Outline):** Escenario que se repite con diferentes datos, definidos en una tabla de `Ejemplos`.

Ejemplo

```
# language: es

Característica: Registro de empleados
  Como administrador del sistema
  Quiero registrar nuevos empleados
  Para iniciar su proceso de onboarding

  Antecedentes:
    Dado que existe un departamento "IT" con nombre "Tecnología"
    Y que estoy autenticado como "ADMIN"

  Escenario: Registro exitoso de un empleado
    Cuando registro un empleado con los siguientes datos:
      | id    | nombre     | email              | departamentoId |
      | E001  | Juan Pérez | juan@empresa.com   | IT             |
    Entonces la respuesta debe tener código 201
    Y el cuerpo debe contener el nombre "Juan Pérez"
```

Framework Cucumber

Cucumber es la herramienta que conecta los archivos `.feature` (Gherkin) con código ejecutable:

```
📄 .feature (Gherkin)  →  🥒 Cucumber (Runner)  →  📊 Reportes
⚙️ Step Definitions (Código)  ↗
```

- Los **archivos** `.feature` describen **qué** debe hacer el sistema (lenguaje natural).
- Los **step definitions** describen **cómo** se ejecuta cada paso (código).
- **Cucumber** lee los `.feature`, ejecuta los step definitions asociados y genera reportes.

Cada línea de un escenario se vincula con un método anotado en código. Por ejemplo, la línea `Dado que estoy autenticado como "ADMIN"` se asocia con:

```
@Dado("que estoy autenticado como {string}")
public void autenticarComo(String rol) {
    // Código que realiza la autenticación
}
```

Clientes HTTP para Step Definitions

Los step definitions deben ejecutar **peticiones HTTP reales** contra el sistema desplegado. Algunas librerías disponibles por lenguaje:

Lenguaje	Librerías HTTP
Java	`java.net.http` (Java 11+), Rest Assured (recomendado para pruebas), OkHttp, Feign
Python	requests, httpx
Node.js	axios, node-fetch
Go	net/http

Ejemplo con **Rest Assured** (Java):

```
@Cuando("consulto la lista de empleados")
public void consultarEmpleados() {
    response = given()
        .header("Authorization", "Bearer " + token)
        .when()
        .get(BASE_URL + "/empleados");
}

@Entonces("la respuesta debe tener código {int}")
public void verificarCodigo(int codigo) {
    response.then().statusCode(codigo);
}
```

Manejo de la Asincronía: Polling

En un sistema basado en eventos, la **eventual consistencia** es un desafío: cuando se crea un empleado, los otros servicios procesan el evento de forma asincrónica. No es seguro consultar inmediatamente el resultado.

Estrategia: Polling con reintentos. Consultar repetidamente hasta que la condición se cumpla o expire un timeout.

```
función esperarHastaQue(condicion, maxIntentos, intervaloMs):
    para i en 1..maxIntentos:
        intentar:
            si condicion() == verdadero:
                retornar éxito
        capturar error:
            // Todavía no está listo, reintentar
        esperar(intervaloMs)
    lanzar TimeoutException("Condición no cumplida después de " + maxIntentos + " inten
```

Parámetro	Valor sugerido	Justificación
Máximo de intentos	10-15	Suficiente para la mayoría de procesamiento de eventos
Intervalo entre intentos	1-2 segundos	Balance entre velocidad y carga
Timeout total	~30 segundos	Límite razonable para procesamiento asincrónico

> **Importante:** No utilice `sleep` fijo (ej. `Thread.sleep(5000)`). El polling es superior porque la prueba termina más rápido si el evento se procesa rápidamente, y tolera variaciones de tiempo sin fallar de forma intermitente.

> **Nota:** Para profundizar en estos conceptos, revise el material de clase sobre BDD.

Requisitos Generales

- Utilizar un framework BDD compatible con Cucumber (o equivalente) según el lenguaje elegido.
- Escribir los escenarios en archivos `.feature` utilizando la sintaxis **Gherkin en español** (`# language: es`).
- Implementar las **definiciones de pasos (step definitions)** que ejecuten peticiones HTTP reales contra el sistema desplegado.
- Las pruebas deben poder ejecutarse con un **único comando** después de levantar el sistema.

Retos

Los siguientes cinco puntos están organizados en orden de complejidad creciente. Se recomienda abordarlos en secuencia, ya que cada punto construye sobre las habilidades y el código del anterior.

Punto 1: Configuración del proyecto de pruebas BDD (1.0 pt)

Lo que se entrega

La siguiente tabla resume los frameworks BDD y librerías HTTP disponibles por lenguaje:

Lenguaje	Framework BDD	Librería HTTP	Comando de ejecución
Java	Cucumber-JVM	Rest Assured / OkHttp / java.net.http	`mvn test`
Python	Behave / pytest-bdd	requests / httpx	`behave` o `pytest`
Node.js	Cucumber.js	axios / node-fetch	`npx cucumber-js`
Go	Godog	net/http	`godog`

> **Referencia:** Tutorial de Cucumber – 10 minutos

Lo que debe hacer

1. **Crear un proyecto de pruebas independiente** (`e2e-tests/` o nombre equivalente) separado del código de los microservicios, con la siguiente estructura base:

```
e2e-tests/
├── features/              ← archivos .feature (Gherkin)
├── step_definitions/      ← implementación de los pasos
├── support/               ← contexto compartido, hooks, utilidades
└── [archivo de proyecto]  ← pom.xml, package.json, requirements.txt, etc.
```

2. **Elegir y configurar** el framework BDD y la librería HTTP que utilizará.
3. **Diseñar el mecanismo de contexto compartido** entre pasos (también llamado "World" en algunos frameworks). Este contexto debe permitir:

- Almacenar el token de autenticación actual
- Almacenar la última respuesta HTTP recibida
- Acceder a la URL base del sistema mediante variable de entorno

4. **Configurar variables de entorno** para que las pruebas no dependan de valores hardcodeados:
   - `BASE_URL` — URL base del sistema (ej. `http://localhost:8080`)
   - Credenciales de los usuarios de prueba (ADMIN, USER)
5. **Verificar la configuración** creando un escenario mínimo de "humo" que confirme que el sistema está operativo:

```
# language: es

Característica: Verificación del sistema
  Escenario: El sistema responde correctamente
    Dado que el sistema está desplegado y operativo
    Entonces la respuesta debe tener código 200
```

Implemente el step definition correspondiente y ejecute la prueba.

Decisión técnica requerida

Usted debe decidir cómo estructurar el contexto compartido. Algunas preguntas que debe resolver:

- ¿Cómo se almacena y reutiliza el token entre pasos `Dado` y `Cuando` ?
- ¿Se crea una nueva instancia de contexto por escenario o se reutiliza?
- ¿Cómo se gestiona la autenticación en los `Antecedentes` ?

Punto 2: Escenarios de seguridad y control de acceso (1.0 pt)

Lo que se entrega

Un escenario de ejemplo que valida el acceso sin token:

```
# language: es

Característica: Seguridad y control de acceso
  Como sistema de autenticación
  Quiero controlar el acceso a los recursos
  Para garantizar que solo los usuarios autorizados realicen operaciones

  Escenario: Acceso denegado sin token de autenticación
    Cuando consulto la lista de empleados sin token de autenticación
    Entonces la respuesta debe tener código 401
```

Lo que debe hacer

1. **Partir del escenario de ejemplo** proporcionado e implementar su step definition.
2. **Escribir al menos 3 escenarios adicionales** que validen las reglas de seguridad RBAC implementadas en el Reto 4. Los escenarios deben cubrir, como mínimo:
   - Acceso con token inválido o malformado
   - Usuario con rol `USER` puede consultar recursos pero **no puede** modificarlos (crear, eliminar)
   - Usuario con rol `ADMIN` puede realizar todas las operaciones
3. **Implementar todos los step definitions** necesarios.
4. **Usar `Antecedentes`** para factorizar las precondiciones comunes entre escenarios del mismo archivo `.feature`.

Por qué este punto va primero que onboarding/offboarding

Los escenarios de seguridad son **100% sincrónicos** (request → response inmediata). Esto le permite enfocarse en aprender la mecánica de Gherkin + step definitions sin la complejidad adicional de la asincronía. Una vez dominados estos escenarios, los siguientes puntos agregan esa capa.

Punto 3: Escenarios de onboarding con verificación asincrónica (1.0 pt)

Lo que se entrega

El **concepto de polling** como estrategia para manejar la eventual consistencia en un sistema de eventos:

Cuando se crea un empleado, eventos asincrónicos disparan acciones en otros servicios (creación de credenciales, envío de notificaciones). No es posible verificar inmediatamente el resultado del evento. La estrategia es **consultar repetidamente** hasta que la condición se cumpla o se agote un timeout.

```
función esperarHastaQue(condicion, maxIntentos, intervaloMs):
    para i en 1..maxIntentos:
        intentar:
            si condicion() == verdadero:
                retornar éxito
        capturar error:
            // Todavía no está listo, reintentar
        esperar(intervaloMs)
    lanzar TimeoutException("Condición no cumplida después de " + maxIntentos + " inten
```

Convención Gherkin: Cuando un paso requiere polling, se usa la palabra **"eventualmente"** para indicar que no se espera resultado inmediato:

```
Entonces eventualmente el servicio de autenticación debe haber creado un usuario para "
```

Lo que debe hacer

1. **Implementar la función de polling** en su lenguaje elegido, basándose en el pseudocódigo proporcionado.
2. **Definir los parámetros de polling** (máximo de intentos, intervalo, timeout total) y **justificar su elección** en el README.
3. **Escribir los escenarios de onboarding** que cubran:
   - Registro exitoso de un empleado → verificar que se generaron credenciales (evento asincrónico)
   - Registro exitoso → verificar que se generó una notificación (evento asincrónico)
   - El nuevo empleado puede establecer su contraseña y hacer login exitosamente
   - Registro con datos inválidos (departamento inexistente, campos faltantes) → error
4. **Implementar todos los step definitions**, usando polling para los pasos que verifican resultados asincrónicos.
5. **Garantizar independencia entre escenarios:** Cada escenario debe poder ejecutarse de forma aislada, sin depender del resultado de otro.

Decisión técnica requerida

- **Aislamiento de datos:** ¿Cómo garantiza que un escenario no deje "basura" que afecte a otro? ¿Usa IDs únicos generados? ¿Limpia el estado en hooks? ¿Usa `Antecedentes` para crear el estado necesario?
- **Calibración del polling:** ¿Qué timeout es razonable para su sistema? ¿Cómo lo calibró?

> **Importante:** No se aceptan `Thread.sleep()` o `time.sleep()` fijos. El polling es superior porque la prueba termina más rápido si el evento se procesa rápidamente, y tolera variaciones de tiempo sin fallar de forma intermitente.

Punto 4: Escenarios de offboarding (1.0 pt)

Lo que se entrega

Nada. Usted diseña estos escenarios con base en lo aprendido en los puntos 2 y 3.

Lo que debe hacer

1. **Diseñar y escribir los escenarios** de desvinculación de empleados, cubriendo al menos:
   - Desvinculación completa: eliminar empleado → verificar notificación asincrónica de tipo desvinculación
   - El empleado desvinculado **no puede** hacer login
   - La recuperación de contraseña **falla** para un empleado desvinculado
2. **Implementar los step definitions**, reutilizando pasos ya creados en los puntos anteriores cuando sea posible.
3. **Manejar las precondiciones complejas:** El escenario de offboarding necesita que exista un empleado con credenciales activas y configuradas. Esto implica que en los `Antecedentes` o en un hook de setup se debe recrear ese estado desde cero.

Decisión técnica requerida

- **Setup complejo:** ¿Cómo prepara el estado inicial (empleado registrado + credenciales activas) sin depender de otros escenarios? Algunas opciones:
  - Usar `Antecedentes` que invoquen los endpoints de creación
  - Usar hooks `Before` que configuren los datos necesarios
  - Usar datos precargados en la base de datos

- **Reutilización de pasos:** ¿Qué pasos de los puntos anteriores puede reutilizar directamente? ¿Cuáles necesitan ser parametrizados?

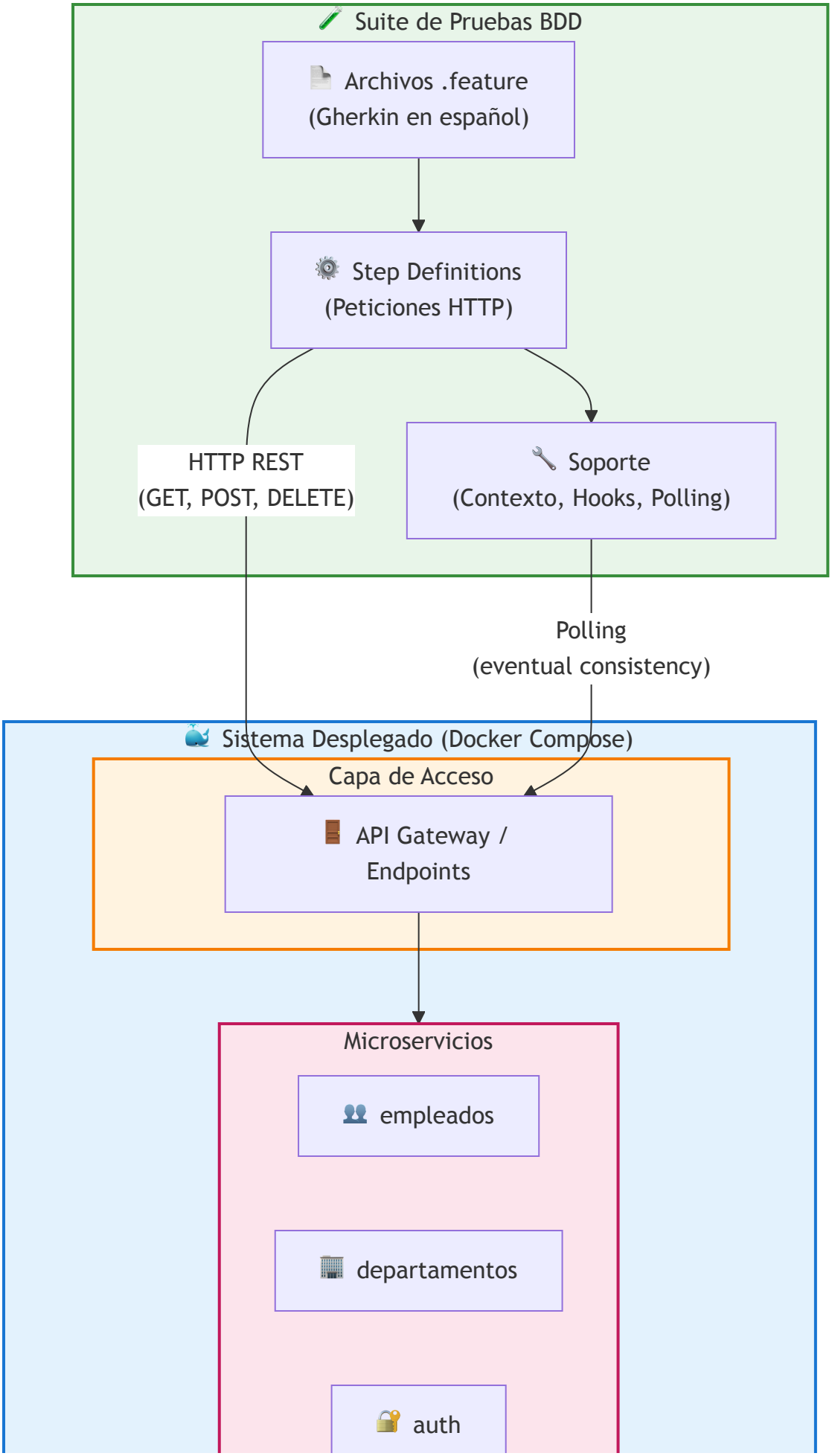
Punto 5: Reproducibilidad y documentación (1.0 pt)

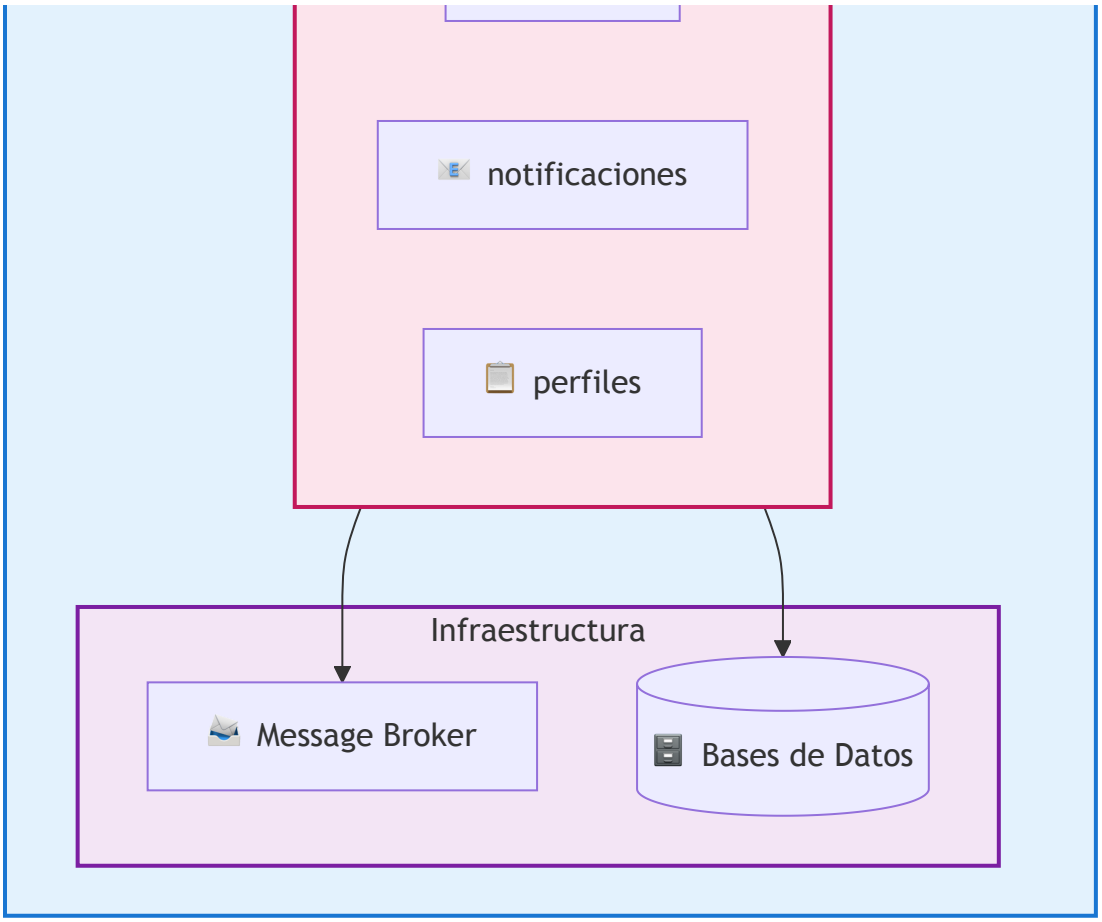
Lo que debe hacer

1. **Ejecución con un único comando:** Toda la suite BDD debe poder ejecutarse con un solo comando después de levantar el sistema con `docker-compose up --build -d`.
2. **Verificar consistencia:** Ejecute la suite completa al menos 3 veces consecutivas y confirme que todos los escenarios pasan de forma consistente. Si hay pruebas intermitentes (*flaky tests*), identifique y corrija la causa.
3. **Simular un fallo:** Modifique un dato esperado en un escenario (ej. cambie un código de respuesta esperado) y verifique que la prueba falla con un **mensaje descriptivo** que indique claramente qué se esperaba y qué se obtuvo.
4. **Documentar en el README.md:**
   - Breve explicación de qué es BDD y por qué se eligió este enfoque
   - **Prerrequisitos** para ejecutar las pruebas (herramientas, versiones)
   - **Instrucciones exactas** de ejecución paso a paso, incluyendo:
     a. Cómo levantar el sistema
     b. Cómo configurar las variables de entorno
     c. Comando para ejecutar las pruebas
     d. Cómo interpretar los resultados
   - **Descripción de los escenarios implementados** (qué flujos se cubren)
   - **Herramientas y frameworks** utilizados con justificación de la elección
5. **(Opcional)** Configurar las pruebas como un **contenedor Docker** dentro del `docker-compose.yml` para que puedan ejecutarse sin instalar dependencias localmente:

```yaml
services:
  bdd-tests:
    build: ./e2e-tests
    depends_on:
      - empleados-service
      - auth-service
      - notificaciones-service
    environment:
      BASE_URL: http://api-gateway:8000
    networks:
      - microservices-network
```

Diagrama de Arquitectura de Pruebas BDD


Suite de Pruebas BDD
Archivos .feature
(Gherkin en español)
Step Definitions
(Peticiones HTTP)
HTTP REST
(GET, POST, DELETE)
Soporte
(Contexto, Hooks, Polling)
Polling
(eventual consistency)
Sistema Desplegado (Docker Compose)
Capa de Acceso
API Gateway /
Endpoints
Microservicios
empleados
departamentos
auth



Entregables

- Proyecto de pruebas BDD con archivos `.feature` y step definitions, versionado en GitHub.
- README.md actualizado según lo especificado en el Punto 5.

Consideraciones

- Los archivos `.feature` deben estar escritos en **español** usando `# language: es`.
- Las step definitions deben usar **variables de entorno** para configurar URLs base y credenciales.
- Cada escenario debe ser **independiente**: no debe depender del resultado de otro escenario.
- Los pasos que esperan resultados de eventos asíncronos deben implementar **polling con reintentos**, no `sleep` fijo.
- Si un escenario crea datos, use identificadores únicos o limpie el estado entre ejecuciones.

Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, correspondiendo **1 punto** a cada uno de los retos:

#	Reto	Valor	Aspectos a evaluar
1	**Configuración del proyecto**	1.0	Proyecto independiente estructura correcta. Contexto compartido entre pasos bien diseñado. Variables de entorno configuradas. Escenario de humo funcional. Justificación del framework elegido.
2	**Escenarios de seguridad**	1.0	Mínimo 4 escenarios (1 dado + 3 propios). Cubren token ausente, token inválido, diferencia USER vs ADMIN. Steps implementados correctamente. Uso adecuado de `Antecedentes`.
3	**Escenarios de onboarding**	1.0	Escenarios cubren registro, creación de credenciales vía evento, login exitoso, y datos inválidos. Polling implementado correctamente (no `sleep` fijo). Parámetros de polling justificados. Escenarios independientes.
4	**Escenarios de offboarding**	1.0	Escenarios diseñados por el estudiante. Cubren desvinculación, bloqueo de login, y fallo de recuperación. Precondiciones complejas resueltas. Reutilización de pasos.
5	**Reproducibilidad y documentación**	1.0	Suite ejecutable con un comando. Resultados consistentes. README completo con instrucciones, descripción de escenarios, y justificación de herramientas.

> **Nota:** Se valorará especialmente la **claridad y expresividad** de los escenarios Gherkin. Un buen escenario debe poder ser entendido por alguien que no conoce el código, actuando como documentación viva del sistema.